

AVALIAÇÃO FINAL

Avaliação Final — Análise, Testes e Refatoração de Código Legado

1. Contexto

Esta avaliação final tem como objetivo aplicar, de forma integrada, os principais conteúdos trabalhados na disciplina **Tópicos Especiais em Computação II**, incluindo qualidade de código, refatoração, testes automatizados, organização arquitetural e uso crítico de inteligência artificial no desenvolvimento de software.

O repositório-base contém um **código legado simulado com débito técnico intencional**, relacionado ao processamento de solicitações de viagem institucional. O código funciona, mas apresenta problemas deliberados de legibilidade, organização, separação de responsabilidades, testabilidade e arquitetura.

A tarefa da equipe é compreender o comportamento existente, preservar esse comportamento por meio dos testes fornecidos, refatorar o código e reorganizar a solução de forma tecnicamente mais adequada.

2. Objetivo da atividade

A equipe deverá:

- analisar o código original fornecido;
- preservar o comportamento funcional existente;
- executar e manter os testes de preservação de comportamento;
- escrever testes de unidade próprios para os objetos implementados;
- refatorar o código com foco em legibilidade, simplicidade e manutenção;
- reorganizar a solução com separação mínima entre domínio, aplicação e infraestrutura;
- implementar persistência simples utilizando o banco fornecido no projeto;
- elaborar um diagrama de dependências em PDF;
- documentar o processo, incluindo o uso crítico de ferramentas de IA.

3. Organização da equipe

A atividade deverá ser realizada individualmente ou em dupla.

O `README.md` deverá conter obrigatoriamente os nomes completos dos membros da equipe.

A identificação da autoria é requisito obrigatório da avaliação. Repositórios cujo `README.md` não contenha os nomes completos dos membros da equipe receberão **nota ZERO**, independentemente da existência de código, testes, commits, diagrama ou demais artefatos entregues.

4. Repositório e forma de entrega

O professor fornecerá um repositório público contendo o código-base da avaliação.

A equipe deverá:

1. clonar o repositório-base;
2. criar um novo repositório público próprio no GitHub;
3. publicar a solução desenvolvida nesse novo repositório;
4. entregar no SIGAA apenas o link do repositório público da equipe.

Não é necessário realizar fork nem abrir Pull Request para o repositório do professor.

5. Código original

O código original está localizado em:

```
src/original/
```

Esse código **deverá ser preservado** no repositório. A equipe não deve apagar a pasta `src/original`.

A versão refatorada deverá ser implementada nas demais pastas do projeto, respeitando a estrutura exigida.

6. Contrato público da aplicação

Os testes de preservação de comportamento utilizarão o contrato público exportado por:

```
src/main.ts
```

A equipe poderá alterar a implementação interna da solução, mas deverá manter compatibilidade com o contrato público esperado pelos testes.

O arquivo `src/main.ts` deverá exportar a função principal da avaliação e os tipos necessários para sua utilização. Esse arquivo será utilizado como ponto público de entrada da aplicação pelos testes de preservação de comportamento.

A função principal da avaliação é:

```
export function processTravelRequest(
  input: TravelRequestInput,
): TravelRequestOutput;
```

A equipe não deve criar nem utilizar `src/index.ts` como ponto de entrada obrigatório da solução. Para esta avaliação, o contrato público deverá estar centralizado em `src/main.ts`.

7. Regras funcionais

A função `processTravelRequest` recebe uma solicitação de viagem institucional e retorna uma análise da solicitação.

A entrada contém os seguintes dados:

```
export type TravelRequestInput = {
  requestId: string;
  requesterName: string;
  requesterType: "student" | "employee" | "professor" | "manager";
  destination: string;
  departureDate: string;
  returnDate: string;
  reason: string;
  transportCostInCents: number;
};
```

A saída esperada segue o formato:

```
export type TravelRequestOutput = {
  requestId: string;
  status: "approved" | "pending-review" | "rejected";
  travelDays: number;
  dailyAmountInCents: number;
  subtotalInCents: number;
  totalAmountInCents: number;
  errors: string[];
  warnings: string[];
};
```

Regras funcionais principais:

- `requestId`, `requesterName`, `requesterType`, `destination`, `departureDate` e `returnDate` são obrigatórios;
- as datas devem estar no formato `YYYY-MM-DD`;
- `returnDate` não pode ser anterior a `departureDate`;
- a quantidade de dias deve ser calculada de forma inclusiva;
- o valor da diária depende do tipo de solicitante:
 - `student`: R\$ 90,00, representado por `9000`;
 - `employee`: R\$ 180,00, representado por `18000`;
 - `professor`: R\$ 250,00, representado por `25000`;
 - `manager`: R\$ 300,00, representado por `30000`;
- `subtotalInCents` deve ser calculado por `travelDays * dailyAmountInCents`;
- `totalAmountInCents` deve ser calculado por `subtotalInCents + transportCostInCents`;
- havendo erro de validação, o status deve ser `rejected`;
- se não houver erro, mas a viagem tiver mais de 5 dias, o status deve ser `pending-review`;
- se não houver erro, mas o valor total for superior a R\$ 2.000,00, o status deve ser `pending-review`;
- se a viagem tiver mais de 5 dias e a justificativa tiver menos de 30 caracteres, deve ser adicionada uma advertência em `warnings`;
- nos demais casos, o status deve ser `approved`.

As mensagens de erro e advertência fazem parte do comportamento público observado pelos testes. Elas devem permanecer estáveis, em inglês e exatamente como definidas abaixo:

```
requestId is required
requesterName is required
requesterType is required
destination is required
departureDate is required
returnDate is required
departureDate must be a valid YYYY-MM-DD date
returnDate must be a valid YYYY-MM-DD date
returnDate cannot be before departureDate
long travel requests should include a detailed reason
```

8. Testes de preservação de comportamento

O repositório contém testes de preservação de comportamento em:

```
tests/original/
```

Esses testes foram fornecidos pelo professor e representam o comportamento observável esperado da aplicação. Eles não existem para validar a forma interna do código, mas para garantir que, depois da refatoração, a função pública continue respondendo da mesma maneira para os cenários cobertos.

A equipe não deve remover, renomear ou modificar os testes localizados em `tests/original`.

A solução final deverá continuar passando nesses testes.

Os testes de preservação de comportamento utilizarão o contrato público exportado por `src/main.ts`. Assim, a equipe poderá reorganizar internamente a solução, desde que mantenha o comportamento esperado pela função `processTravelRequest`.

No repositório-base, `src/main.ts` aponta para a implementação legada em `src/original/`. Isso permite que a equipe execute os testes antes de qualquer alteração e observe a linha de base do comportamento esperado.

Durante a atividade, a equipe deverá implementar o novo código nas pastas adequadas, como `src/domain/`, `src/application/` e `src/infra/`, mantendo o contrato público de `src/main.ts`. O intercâmbio entre o código original e o código produzido deverá acontecer nesse ponto público de entrada:

1. inicialmente, `src/main.ts` expõe a função legada;
2. após criar a solução refatorada, a equipe altera `src/main.ts` para delegar a chamada à nova implementação;
3. os testes em `tests/original/` continuam importando exclusivamente de `src/main.ts`;
4. se os testes continuarem passando, há evidência de que o comportamento público coberto foi preservado.

Portanto, os testes de preservação de comportamento não devem importar diretamente de `src/original/`, nem de módulos internos da solução refatorada. Eles devem permanecer acoplados somente ao contrato público da aplicação. Esse desenho permite trocar a implementação interna sem trocar os testes públicos.

Os testes fornecidos cobrem cenários essenciais, mas não substituem os testes próprios da equipe. Eles protegem o comportamento legado conhecido; os testes próprios devem validar os novos objetos, regras, casos de uso e componentes de persistência criados pela equipe.

9. Testes próprios da equipe

Além dos testes fornecidos pelo professor, a equipe deverá escrever testes de unidade para os objetos implementados.

Os testes próprios deverão seguir, dentro de `tests`, a mesma organização adotada em `src`.

Exemplo:

```
src/original/  
tests/original/  
  
src/domain/  
tests/domain/  
  
src/application/  
tests/application/  
  
src/infra/  
tests/infra/
```

Devem ser testados os objetos que contenham comportamento relevante, tais como regras de negócio, validações, cálculos, casos de uso e componentes de persistência.

Não é necessário criar testes para DTOs, interfaces, tipos sem comportamento próprio ou arquivos meramente declarativos.

10. Persistência

O repositório-base fornece uma infraestrutura simples de banco de dados.

A equipe deverá implementar persistência para salvar e recuperar solicitações ou análises de viagem.

O uso de ORM é opcional. A equipe poderá utilizar ORM, query builder, SQL direto ou driver nativo.

O banco de dados deve ser tratado como detalhe de infraestrutura. As regras de negócio e o caso de uso principal não devem depender diretamente de detalhes específicos do banco.

11. Estrutura obrigatória de pastas

A solução deverá respeitar a seguinte estrutura mínima:

```
src/  
  original/  
  domain/  
  application/  
  infra/  
  main.ts  
  
tests/  
  original/  
  domain/  
  application/  
  infra/  
  
docs/  
  tec2-aval.md  
  dependency-diagram.pdf
```

Arquivos e subpastas adicionais poderão ser criados, desde que respeitem a separação entre domínio, aplicação, infraestrutura, testes e documentação.

O arquivo `src/main.ts` deverá ser mantido como ponto público de entrada da aplicação, exportando o contrato utilizado pelos testes de preservação de comportamento.

12. Diagrama de dependências

A equipe deverá entregar um diagrama de dependências em PDF no seguinte caminho:

```
docs/dependency-diagram.pdf
```

O diagrama deverá seguir o estilo do exemplo apresentado pelo professor e representar, no mínimo:

- camada de domínio;
- camada de aplicação;
- camada de infraestrutura;
- caso de uso principal;

- objetos de domínio relevantes;
- interfaces, repositórios, DTOs ou tipos relevantes;
- dependência externa do banco de dados;
- direção das dependências entre os componentes.

O objetivo do diagrama é demonstrar a organização arquitetural da solução refatorada.

13. Uso de inteligência artificial

O uso de ferramentas de inteligência artificial é permitido.

Entretanto, a equipe deverá documentar no `README.md` :

- quais ferramentas foram utilizadas;
- como a IA foi utilizada;
- quais sugestões foram aceitas;
- quais sugestões foram rejeitadas ou modificadas;
- como a equipe validou tecnicamente as respostas geradas.

O uso de IA não substitui a responsabilidade técnica da equipe pela solução entregue.

14. Histórico de commits

O repositório deverá conter histórico de commits compatível com o desenvolvimento incremental do trabalho.

A equipe deverá realizar pelo menos 5 commits significativos, com mensagens em inglês.

Exemplos de mensagens adequadas:

```
test: add domain unit tests
refactor: extract travel period validation
refactor: create travel request use case
feat: add travel request persistence
docs: add dependency diagram
docs: update setup instructions
```

Commits genéricos, artificiais ou concentrados em uma única entrega final poderão receber desconto no critério correspondente.

15. Convenções obrigatórias

A solução deverá respeitar as seguintes convenções:

- utilizar `camelCase` para variáveis, funções, métodos, propriedades, parâmetros e constantes locais;
- utilizar `PascalCase` para classes, interfaces, tipos e enums;
- utilizar `kebab-case` para arquivos e diretórios;
- escrever identificadores de código, comentários, testes, mensagens de commit, logs, erros e textos internos em inglês;
- manter em inglês os nomes relacionados ao código, incluindo variáveis, funções, métodos, classes, interfaces, tipos, enums, arquivos, diretórios, scripts, comandos, variáveis de ambiente, objetos de banco de dados, APIs, rotas, eventos, DTOs, schemas, use cases, repositories, services, configuration keys e quaisquer outros nomes técnicos.

16. Critérios de avaliação

A nota será atribuída conforme os seguintes critérios:

Critério	Peso
Testes e preservação do comportamento	3,0
Refatoração, arquitetura, persistência e diagrama	4,0
README, execução e uso crítico de IA	2,0
Histórico de commits	1,0
Total	10,0

Testes e preservação do comportamento — 3,0

Avalia se a equipe manteve os testes de preservação de comportamento funcionando, criou testes próprios relevantes e preservou o comportamento funcional esperado.

Refatoração, arquitetura, persistência e diagrama — 4,0

Avalia a melhoria do código, a separação de responsabilidades, a organização entre domínio, aplicação e infraestrutura, a implementação da persistência e a qualidade do diagrama de dependências.

README, execução e uso crítico de IA — 2,0

Avalia a clareza da documentação, a presença obrigatória dos nomes dos membros, as instruções de execução, a explicação das decisões técnicas e o registro crítico do uso de IA.

A ausência dos nomes completos dos membros da equipe no `README.md` implicará **nota ZERO** na avaliação, independentemente da pontuação eventualmente obtida nos demais critérios.

Histórico de commits — 1,0

Avalia se o desenvolvimento foi incremental, rastreável e compatível com uma produção efetiva da equipe.

17. Requisitos de execução

O projeto utiliza Node.js 22, TypeScript em modo estrito, npm e Vitest. Ele deverá permitir, no mínimo:

```
npm install
npm run typecheck
npm test
```

Caso o banco de dados seja necessário para executar parte dos testes ou da aplicação, o `README.md` deverá informar os comandos necessários para iniciar e preparar o ambiente.

O repositório-base já fornece scripts npm para a infraestrutura de banco de dados:

```
npm run db:up
npm run db:init
npm run db:down
```

18. Entrega

A entrega deverá ser feita pelo SIGAA, informando exclusivamente o link do repositório público da equipe.

O repositório deverá conter:

- `README.md` completo, contendo obrigatoriamente os nomes completos dos membros da equipe;
- código original preservado;
- solução refatorada;
- testes de preservação de comportamento mantidos;
- testes próprios da equipe;
- persistência implementada;
- diagrama de dependências em PDF;
- histórico de commits adequado.

A ausência dos nomes completos dos membros da equipe no `README.md` implicará **nota ZERO** na avaliação, ainda que os demais requisitos tenham sido atendidos.